

Deep Learning Coding Test

The data

The data you have been provided is not for general use. It is to be used for this coding test only, and must be deleted after you have submitted. Do not share the data with anyone.

Download your data from <https://www.dropbox.com/sc/fj/wng5l2cbmy8kpbjbgcps52/MLtestdata.zip?dl=0&nkey=qo0l0Qrs9c2bmhp1utn7nd620>. It is ~0.6 GB in size.

The password is 'a1s2d3f4'

The dataset consists of images (jpeg files), and masks (npz files).

Images

The images are all aerial photographs. They may not all be the same size, but most of them will be 896x896.

Labels/Masks

The masks (npz files) all match the dimension of their corresponding image, and represent labels for pixels in the image. In other words, `mask[i,j]` is a label (for some class) for pixels in the image. Masks have value 0 (not present) or -1 (unknown/no label), or ≥ 1 (present).

Note that many of the values in the mask are -1.

Masks ending in "_2.npz" are for roofs, masks ending in "_3.npz" are for solar panels.

Note that not all images have labels, and even if labels are available, some images have labels for only roof or only solar (i.e. not both). Many images do have labels for both though.

You can read the mask for an image like this: `mask = np.load(maskfile)['dkmask']`

Data quality

The data is real data, with everything that that implies: there may be bad (fuzzy) images, and there will be some mistakes/noise in the labels. That said, we are not out to trick you: the data is in general of reasonable quality, so do not expect pathologically bad data.

Your task

Please train a neural-network to detect roofs and solar panels. Specifically, we would like your model to tag **every pixel** in an image and say what it contains (does it contain a solar panel? Or a roof? Or both? Or neither?)

Pytorch is our preferred deep learning framework, but you can also use TensorFlow/Keras.

We must stress that we do not expect you to spend time coming up with funky network design or architectures: either develop a very simple network of your own in keras/torch, or feel free to steal/adapt any that is publicly available (if you do this, please reference what you used). We strongly recommend that you err towards using a simple network, so that you can get something done quicker: previous applicants that have tried using complicated networks have struggled to produce a working solution.

Please make note of the major choices you make (architecture, training process, etc). Please include all your code and output in your solution (including data-prep and checking, model training, etc). Please also include some material to indicate how your model performs. Provide at least some basic performance metrics (accuracy, etc) and a few example images with the corresponding predictions from your model.

We've given you a small dataset, and realistically, you do not have time to train a proper deep learning model that performs well, so we do not expect your model to perform well. You will not be marked/ranked on your model prediction metrics. It is sufficient for you to show how you would set up the problem, and show that the loss is going down over a few epochs. We are interested in your general approach, and will take it for granted that you could improve things a lot if you had more time and data. Focus on doing something simple and sensible, rather than trying to get a complicated model: we want to see how you approach the problem rather than see you spend hours tweaking and training.

Feel free to say a few words about what you would do to optimize performance if you had (lots) more time and data (i.e. if this was your job rather than a take home test).

Even though we do not expect your model to perform well, we would like to see some code that shows your model outputs/predictions, and calculates some performance metrics.

Submission

DO NOT publicly fork this repository or create pull requests on it as we don't want other candidates to see your solution. This coding test is confidential and you are required to not share either the data or problem statement with anyone else but Nearmap. You are also required to keep your solution confidential and not share it with anyone outside Nearmap.

Please create a **private** repo in your github account.

Please commit all your code and notes related to this solution in your repo. Also, please make note of the major choices you make (tools, techniques, designs & assumptions).

Once completed, please add the github account mentioned in the email correspondence as a collaborator to your private repo so we can see your solution.

Important Notes

Since many people do not have access to a GPU, we are fine if you just use the CPU for training. You won't get extra credit for having access to a GPU and training more epochs than someone else who just trains on a CPU and submits a model that is a long way from convergence. In other words, we do not expect you to submit a converged model, but rather that you submit something that shows how you would approach the problem. Bad predictions from an un-converged model are perfectly acceptable.

Suggestions

We advise you to stick to core libraries (keras, tensorflow, pytorch, pillow/PIL, skimage), but you are free to use other libraries.

Starting code

Below is some code that may be helpful to get you started thinking about the problem, but which you do not need. Feel free to delete it before your submission.

Note that the data shown in the image below is typical of the data you have -- most of the data consists of -1's (not labelled): only a small proportion of the pixels are actually labelled.

```
import os
import numpy as np
from PIL import Image
from matplotlib import pyplot as plt

# get the image
img = np.asarray(Image.open("MLtestdata/area_6254_2017-08-28.jpg"))

# get the roof and solar mask associated with that image
roofmask = np.load("MLtestdata/area-6254-date-2017-08-28-labels-all-instance- mask_dk_2.npz")['dk']
solarmask = np.load("MLtestdata/area-6254-date-2017-08-28-labels-all- instance-mask_dk_3.npz")['d']

# let's see how they all line up. Remembering that pixels for the labels in the npz files can be
#0 (not present) or (-1) not labelled
f, ax = plt.subplots(nrows=3, ncols=1, figsize=(20,60))
ax[0].imshow(img)
ax[1].imshow(roofmask, vmin=-1.0, vmax=1.0)
ax[2].imshow(solarmask, vmin=-1.0, vmax=1.0)
plt.show()
```

